CUSTOMER SUPPORT MULTI-AGENT AI ASSISTANT

Complete Project Roadmap & Implementation Blueprint

Project goal
Build a portfolio-ready GenAI customer-support application inspired by Microsoft's Customer Chatbot Solution Accelerator, using a custom implementation and a technology stack you already know.

FRONTEND	BACKEND & AI	DATA & RETRIEVAL
React + Vite Responsive chat UI	Python + FastAPI LangChain + LangGraph	MySQL ChromaDB + Gemini embeddings
UI state and API calls	Groq-hosted LLM Pydantic + SQLAlchemy	Git + GitHub Pytest

What you will build

- Product search and product details using MySQL.
- Policy Q&A; for returns, refunds, shipping, warranty and FAQs using RAG.
- Specialized Product & Order and Knowledge agents coordinated by LangGraph.
- Support ticket creation and status lookup through controlled tools.
- Conversation context, source references, React chat UI and FastAPI APIs.
- Testing, security checks, evaluation, documentation and deployment preparation.

Learning approach: implement one layer at a time, test it, understand the design choices, and then move forward. This is an implementation guide—not a requirement to copy Microsoft's full cloud architecture.

Estimated duration: 30–35 days at around 2–3 focused hours daily. Treat this as a planning estimate, not a deadline.

Document Guide

Section	Contents
01	Project overview, objectives and scope
02	Technology stack and responsibilities
03	Architecture and end-to-end request flow
04	Functional requirements and user stories
05	Roadmap overview and phase gates
06	Detailed implementation phases 1–8
07	MySQL database design
08	RAG pipeline and knowledge-base plan
09	LangGraph multi-agent design
10	Recommended folder structure
11	Testing, evaluation, security and deployment
12	Interview preparation and portfolio checklist
13	First milestone and definition of done

01. Project Overview, Objectives and Scope

Scenario: a customer asks, “My headphones stopped working after five days. Can I return them? Also, show me wireless headphones under ■3,000.” The application needs to retrieve policy evidence, search structured product data, and combine the results into one useful answer.

Primary objectives

- Demonstrate full-stack development with React, Python and MySQL.
- Demonstrate retrieval-augmented generation grounded in a business knowledge base.
- Demonstrate controlled tool calling and LangGraph orchestration.
- Build reliable API boundaries, validation, error handling and tests.
- Produce a GitHub repository and demo that can be explained in interviews.

Scope boundaries

- Use mock product, order and customer data during development.
- Do not connect to a real store, payment system or live refund mechanism.
- Require authentication and authorization before exposing customer-specific orders or tickets.
- Use explicit, validated backend tools; do not allow the LLM to execute arbitrary SQL.
- Start with two agents; add more only when the workflow shows a clear need.

02. Technology Stack and Responsibilities

Layer	Technology	Responsibility	Why it is used
Frontend	React + Vite	Chat, messages, loading, citations and errors.	Familiar full-stack UI stack.
API	Python + FastAPI	REST endpoints, request validation and session routing.	Clear boundary between UI and AI logic.
LLM framework	LangChain	Prompts, model calls, retrievers, tools and structured outputs.	Reusable integration patterns.
Orchestration	LangGraph	State, nodes, conditional edges, execution control and checkpoints.	Explicit, testable multi-step workflow.
LLM provider	Groq-hosted model	Natural-language generation and tool calling where supported.	Fast inference; model choice must support needed features.
Relational DB	MySQL + SQLAlchemy	Products, customers, orders, tickets and chat metadata.	Structured, queryable source of truth.
Vector store	ChromaDB	Semantic retrieval over policy/FAQ passages.	Separate retrieval index for unstructured documents.
Embeddings	Gemini embedding API	Convert document chunks and queries into vectors.	Semantic similarity search.
Validation	Pydantic	Validate request and tool input/output schemas.	Typed boundaries and clearer errors.
Testing	Pytest + API client	Unit, integration and workflow tests.	Catch regressions and unsafe behavior.
Version control	Git + GitHub	History, branches, issues and portfolio.	Reproducibility and collaboration.

Data ownership rule: MySQL is the source of truth for structured business data. ChromaDB is an index for semantic document retrieval. Store references and metadata so answers can be traced to their evidence.

03. Architecture and End-to-End Request Flow

React frontend Chat UI, messages, source links, loading and error states
↓ HTTP request / response
FastAPI backend Authentication integration, input validation, conversation/session handling
↓
LangGraph orchestrator Shared state, routing, conditional edges, execution limits
■ ■
Product & Order Agent Controlled tools → MySQL
Knowledge Agent Retriever → ChromaDB → policy evidence
↓
Response assembly Combine results, validate evidence, return answer and references

Request lifecycle

- React submits a message plus a conversation identifier to FastAPI.
- FastAPI validates the request and verifies that the user may access the conversation.
- LangGraph initializes or resumes workflow state and chooses the next node.
- The selected agent invokes a specific tool: SQL-backed lookup, ticket operation or document retrieval.
- The tool returns structured results; the LLM produces a response constrained by those results.
- The backend stores the message/metadata as configured and returns the answer, sources and status to React.

Core design rules

- The LLM does not directly execute arbitrary SQL or receive unrestricted database credentials.
- Order and customer tools enforce authorization in backend code, not only in prompts.
- Policy answers should include retrieved source references where possible and say when evidence is insufficient.
- Set timeouts, maximum tool iterations and safe fallback paths to prevent runaway workflows.
- Keep provider-specific model code behind a small service layer so providers can be changed later.

04. Functional Requirements and User Stories

ID	User story / requirement	Acceptance condition
FR-01	As a customer, I can search products by keyword, category and price.	Results come from MySQL; price filters are applied by SQL, not inferred by the LLM.
FR-02	I can ask questions about returns, refunds, shipping and warranty.	Answer uses relevant retrieved passages and displays source references.
FR-03	I can ask about my order status.	Only the authenticated/authorized customer can access the order.
FR-04	I can create a support ticket.	Validated ticket is persisted; ticket ID and status are returned.
FR-05	I can ask a question that needs product data and policy evidence.	Graph invokes both required paths and assembles a coherent answer.
FR-06	I can continue a conversation.	Conversation context is scoped to the correct session/user.
FR-07	I receive a useful response when information is missing.	System asks a clarification or acknowledges missing evidence.
FR-08	The system handles failures safely.	No secrets or stack traces are exposed; errors are logged with request IDs.

05. Roadmap Overview and Phase Gates

Phase	Indicative days	Primary deliverable	Gate: do not proceed until…
1. Setup	1–2	React ↔ FastAPI connection	Health endpoint works from the frontend.
2. MySQL/API	3–6	Schema and CRUD/search APIs	Sample queries and validation tests pass.
3. LangChain tools	7–10	LLM calls controlled tools	Tool calls handle invalid/missing inputs.
4. RAG	11–15	Grounded policy Q&A;	Retrieval and citations tested on sample questions.
5. LangGraph	16–21	Two-agent orchestration	Routing, mixed questions and failures tested.
6. React UI	22–25	End-to-end chat application	UI handles normal responses, loading and errors.
7. QA/security	26–29	Tests and safeguards	Critical authorization and regression tests pass.
8. Deploy/portfolio	30–35	Demo, README and deployment	Fresh setup is reproducible from documentation.

Phase 1 — Project planning and environment setup (Days 1–2)

Goal: establish a clean, working full-stack development environment.

- Create the repository and top-level frontend/backend folders.
- Create React using Vite; confirm the app runs locally.
- Create a Python virtual environment and install FastAPI plus an ASGI server.
- Implement GET /health returning a small JSON status response.
- Call /health from React and configure CORS for the development origin.
- Add .gitignore and .env.example; ensure actual credentials are never committed.

Deliverable: React can call FastAPI and display a healthy status. **Interview topics:** REST, HTTP, CORS, virtual environments and environment variables.

Phase 2 — MySQL database and backend APIs (Days 3–6)

Goal: implement reliable, validated business data access before adding AI.

- Design normalized tables and relationships for customers, products, orders, order_items, support_tickets, conversations and messages.
- Create schema migrations or a reproducible SQL schema; add indexes for frequent lookups.
- Configure SQLAlchemy sessions and connection settings.
- Implement product search with keyword/category/price filters and bounded pagination.
- Implement order status and ticket endpoints with ownership checks.
- Use Pydantic request/response schemas and consistent error responses.
- Test endpoints using Swagger UI/API client and add pytest coverage.

Deliverable: API endpoints return correct records from MySQL. **Interview topics:** normalization, joins, indexes, transactions, ORM, validation and SQL injection.

Phase 3 — LLM integration and LangChain tools (Days 7–10)

Goal: let the LLM call narrowly scoped, validated business functions.

- Configure the selected Groq-hosted model through LangChain; keep credentials in environment variables.
- Create a model/service wrapper and prompts with clear responsibilities and output expectations.
- Implement tools such as search_products, get_order_status and create_support_ticket.
- Give tools typed arguments and validate identifiers, limits and allowed operations.
- Test direct tool calls first, then let the model select tools for natural-language questions.

- Handle rate limits, timeouts, unavailable models and tool exceptions without leaking secrets.

Deliverable: a model can choose and invoke approved tools and explain their results. **Interview topics:** tool calling vs. text generation, structured outputs, prompts, model choice and hallucinations.

06. Detailed Implementation Phases (continued)

Phase 4 — RAG and knowledge base (Days 11–15)

Goal: answer policy questions from approved source documents rather than relying on model memory.

- Create versioned sample documents for returns, refunds, shipping, warranty and FAQs.
- Load files, normalize text and split documents into meaningful chunks with metadata.
- Generate embeddings with the Gemini embedding API and store vectors in ChromaDB.
- Build a retriever that returns top-k passages and source metadata.
- Pass retrieved context to the LLM with instructions to use the evidence and acknowledge gaps.
- Return citations/source names in the API response and render them in React later.
- Evaluate retrieval using representative questions, including questions absent from the documents.

Deliverable: policy answers with evidence references and a safe fallback. **Interview topics:** embeddings, chunking, overlap, similarity, top-k, metadata filters, retrieval quality and RAG vs. fine-tuning.

Phase 5 — Multi-agent system with LangGraph (Days 16–21)

Goal: create a controlled workflow that routes tasks to specialized responsibilities.

- Define typed/shared graph state: user message, conversation ID, intent, tool results, retrieved sources, final answer and errors.
- Create an orchestration/router node that identifies whether the request needs product/order tools, policy retrieval or both.
- Create a Product & Order Agent with only the business tools it needs.
- Create a Knowledge Agent with the policy retriever and evidence-aware response instructions.
- Connect nodes using conditional edges; define start/end paths and a maximum execution budget.
- For mixed requests, run both required paths and combine the results in a response-assembly node.
- Add checkpointing only after the basic graph works; scope checkpoint IDs to the correct user/session.
- Test ambiguous requests, empty results, tool failures, conflicting evidence and unsupported policy questions.

Deliverable: two specialized agents coordinated by LangGraph. **Interview topics:** graph state, nodes, edges, conditional routing, checkpoints, handoffs and when a single agent is better.

Phase 6 — React chat interface (Days 22–25)

Goal: make the AI workflow usable through a clean customer-facing UI.

- Create reusable components for chat messages, composer, product results, source list and loading/error states.
- Build a frontend API service that sends messages to FastAPI.
- Pass a conversation ID and display prior messages from the current session.
- Render product data as structured UI rather than burying all details in prose.
- Display policy citations with source titles and relevant excerpts when available.
- Handle empty input, duplicate submissions, network failures and long-running requests.
- Verify session separation and test the full flow from React through the database/retriever and back.

Deliverable: a working end-to-end customer chat. **Interview topics:** React hooks, state, async requests, API service layers, CORS and session handling.

Phase 7 — Testing, evaluation and security (Days 26–29)

Goal: verify that the system is reliable, measurable and safe for its intended demo scope.

- Write unit tests for database services, tools, schemas and retrieval functions.
- Write API integration tests and graph workflow tests with deterministic/mock model outputs where possible.
- Build a 30–50 question evaluation set: direct lookups, policy questions, mixed requests, ambiguous inputs and out-of-scope questions.
- Track routing accuracy, retrieval success, answer correctness/faithfulness, latency and approximate token/API cost.

- Check customer/order authorization in backend code and prevent cross-session access.
- Add request IDs, useful logs, timeouts, input length limits and safe public error messages.
- Test prompt injection in documents/user messages; treat retrieved content as untrusted data.
- Avoid logging secrets or unnecessary personal data; document retention and deletion assumptions.

Deliverable: tests, evaluation results and a documented limitations section. **Interview topics:** unit vs. integration tests, evaluation design, observability, prompt injection, privacy and latency/cost trade-offs.

Phase 8 — Deployment and portfolio preparation (Days 30–35)

Goal: make the project reproducible and easy for a reviewer to understand.

- Move all environment-specific settings into environment variables and document required keys without values.
- Prepare deployment configuration for React, FastAPI, MySQL and ChromaDB; verify provider/network constraints.
- Use HTTPS and restrict CORS to the deployed frontend origin.
- Run database migrations or initialization scripts against a clean environment.
- Create a professional README: problem statement, features, architecture, setup, environment variables, tests, limitations and screenshots.
- Add an architecture diagram, sample API requests/responses and a short demo video.
- Prepare a concise walkthrough of one policy query, one product query and one mixed multi-agent query.

Deliverable: a portfolio-ready repository and demo. **Interview topics:** deployment architecture, secrets management, scalability, cost and model/provider trade-offs.

07. MySQL Database Design

Table	Important fields (illustrative)	Purpose
customers	id, name, email, created_at	Customer identity; keep sensitive fields minimal.
products	id, name, description, category, price, stock, active	Catalog and price/stock lookup.
orders	id, customer_id, status, created_at	Order status; access must be authorized.
order_items	id, order_id, product_id, quantity, unit_price	Products and quantities for each order.
support_tickets	id, customer_id, order_id, subject, status, priority, created_at	Support issue creation and tracking.
conversations	id, customer_id/session owner, created_at	Chat session ownership and metadata.
messages	id, conversation_id, role, content, created_at	Persist user and assistant transcript if enabled.

Design notes

- Use primary and foreign keys; index common lookup fields such as product category, price, order customer_id and ticket status.
- Use transactions for multi-step writes such as creating a ticket and associated events.
- Use parameterized queries/ORM methods; never interpolate user text into SQL.
- Apply authorization in service/tool code. Knowing an order ID must not grant access to that order.
- Conversation transcripts and LangGraph checkpoints are different: transcripts are user-visible history; checkpoints store workflow state needed to resume execution.

08. RAG Pipeline and Knowledge-Base Plan

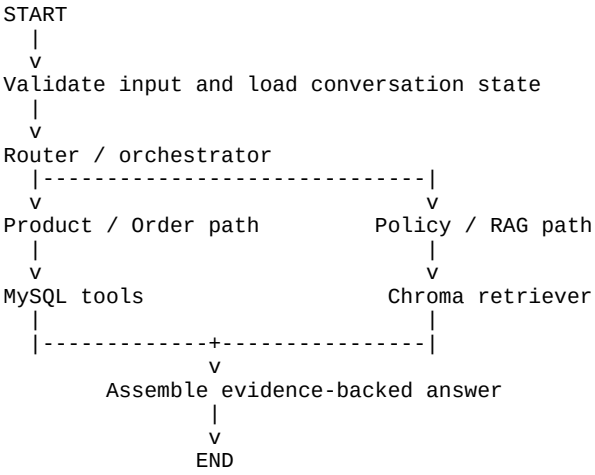
Step	What happens	Validation
1. Ingest	Load policy/FAQ files and capture source metadata/version.	No empty or unsupported files.
2. Split	Chunk by meaningful sections; tune size/overlap based on results.	Avoid broken sentences or chunks without context.
3. Embed	Create vectors with the selected embedding model.	Embedding model/dimensions consistent for indexing and queries.
4. Index	Store vectors and metadata in ChromaDB.	Can retrieve expected source passages.
5. Retrieve	Retrieve top-k relevant chunks; optionally filter by document type/version.	Measure whether expected evidence appears.
6. Generate	Give retrieved evidence to the LLM with grounding instructions.	Answer matches evidence and acknowledges missing information.
7. Cite/evaluate	Return source titles/IDs and evaluate a fixed question set.	Citations support claims; unsupported prompts do not invent policy.

09. LangGraph Multi-Agent Design

Start with a small graph; increase complexity only when tests show the workflow needs it.

Node / component	Responsibility
Input / normalize	Validate message, conversation ID and any request metadata.
Router / orchestrator	Decide whether product/order tools, policy retrieval or both are needed.
Product & Order Agent	Use controlled MySQL-backed tools and return structured facts.
Knowledge Agent	Retrieve policy passages and draft an evidence-grounded answer.
Response assembly	Combine agent outputs, preserve source references and flag missing results.
Validation / finish	Check output shape, enforce limits and return the final response.

Illustrative workflow



State should carry only the information the graph needs, such as the current request, user/session reference, route decision, tool results, retrieved passages, final answer and error status. Do not put API keys or unnecessary personal data in graph state.

Multi-agent design trade-off

Specialized agents can improve separation of responsibilities and make tools easier to scope, but they add latency, cost and debugging complexity. Use a single-agent or direct deterministic path for simple tasks where that is sufficient. Demonstrate that the orchestrator chooses a useful path rather than forcing every request through every agent.

10. Recommended Project Folder Structure

```
customer-support-ai/
■■■ frontend/
■   ■■■ src/
■   ■   ■■■ components/
■   ■   ■■■ pages/
■   ■   ■■■ services/
■   ■   ■■■ App.jsx
■   ■■■ package.json
■■■ backend/
■   ■■■ app/
■   ■   ■■■ api/
■   ■   ■■■ agents/
■   ■   ■■■ graph/
■   ■   ■■■ tools/
■   ■   ■■■ rag/
■   ■   ■■■ database/
■   ■   ■■■ schemas/
■   ■   ■■■ services/
■   ■   ■■■ main.py
■   ■■■ tests/
■   ■■■ requirements.txt
■   ■■■ .env.example
■■■ data/
■   ■■■ knowledge_base/
■■■ docs/
■   ■■■ architecture.md
■■■ .gitignore
■■■ README.md
```

Keep API routes, database operations, agents, graph orchestration and RAG code in separate modules. Add modules gradually as they become necessary; avoid creating empty layers that add no value.

11. Testing, Evaluation, Security and Deployment

Testing layers

- Unit tests: database services, tools, schemas, text chunking and retrieval wrappers.
- API integration tests: validation, expected status codes, persistence and authorization.
- Graph tests: route selection, mixed queries, tool errors, missing evidence and execution limits.
- End-to-end tests: React → FastAPI → graph → tool/retriever → final UI response.
- Regression set: keep fixed questions and expected behavior so prompt/model changes can be compared.

Evaluation metrics

Metric	What it tells you
Routing accuracy	Whether the correct workflow path/agent was selected.
Retrieval success	Whether relevant evidence appeared among retrieved chunks.
Answer correctness / faithfulness	Whether the response matches known facts and retrieved evidence.
Citation support	Whether displayed sources actually support the associated claims.
Latency and cost	Response time and approximate model/API usage per request.
Safe fallback rate	Whether unsupported questions are handled without fabrication.

Security checklist

- Keep API keys and database credentials in environment variables or a secrets manager.
- Validate user input and constrain tool arguments, query limits and model output sizes.
- Enforce authentication and authorization on the backend; never trust user-provided customer IDs.
- Treat user messages and retrieved documents as untrusted content; defend against prompt injection.
- Use least-privilege database credentials, parameterized queries and safe error messages.

- Avoid logging secrets and unnecessary personal data; configure retention intentionally.

Deployment checklist

- Document and verify a clean installation from scratch.
- Configure production environment variables, HTTPS and restrictive CORS.
- Provision MySQL and persistent storage for ChromaDB as required by the deployment setup.
- Run schema setup/migrations and smoke tests in the target environment.
- Add health checks, logs and basic monitoring; document known limits and costs.

12. Interview Preparation and Portfolio Checklist

Topic	Questions to prepare
LLM fundamentals	What are tokens, temperature, context windows and hallucinations? How do you choose a model?
Embeddings	How are text embeddings used? Why must indexing and query embeddings be compatible?
RAG	Why RAG instead of fine-tuning? How do chunking and top-k affect results? How do you ground/cite answers?
Vector stores	What does semantic similarity search do? Why use ChromaDB separately from MySQL?
Tools	How does tool calling work? How are arguments validated? Why not let the LLM execute arbitrary SQL?
LangGraph	What are state, nodes, edges, conditional routing and checkpoints?
Multi-agent	Why use multiple agents? When would one agent or deterministic code be better?
Backend/DB	Explain FastAPI, Pydantic, SQLAlchemy, joins, indexes, transactions and authorization.
Production	How do you measure correctness, latency and cost? How do you handle injection and data leakage?

Portfolio deliverables

- Working application with at least two specialized agents and an end-to-end React interface.
- GitHub repository with meaningful commits and no secrets.
- README with overview, features, architecture, stack, setup, environment variables, test commands and limitations.
- Architecture diagram and example request/response flow.
- Evaluation results from 30–50 representative test questions.
- Demo video showing a product query, policy query and mixed multi-agent request.
- A clear explanation of a bug or design challenge you faced and how you resolved it.

13. Definition of Done

Checklist item	Complete when…
Full-stack integration	React can send a message and display the FastAPI response.
Structured data	Product/order/ticket operations use validated MySQL-backed services.
RAG	Policy questions retrieve relevant sources and unsupported queries do not invent policy.
Agents	LangGraph routes to specialized agents and handles mixed requests.
Security	Customer data access is authorized; secrets are not committed or exposed.
Quality	Tests cover core paths, failures and key evaluation questions.
Portfolio	README, architecture diagram, setup instructions and demo are available.

14. First Milestone: Start Here

- Create the top-level customer-support-ai directory and initialize Git.
- Create frontend with Vite and verify it runs locally.
- Create backend virtual environment and install FastAPI plus Uvicorn.
- Implement GET /health and return a small JSON status.
- Call the health endpoint from React; configure development CORS.

- Add .gitignore and .env.example; confirm no credentials are tracked.
- Commit the working baseline before moving to MySQL.

Phase 1 is complete only when: the frontend calls the running backend successfully, the health status is visible in the UI, and the project can be started again using documented commands.

Recommended working rhythm

For every phase: (1) learn the concept, (2) design the flow, (3) implement small, commented steps, (4) run tests and inspect errors, (5) write interview notes, and (6) review the phase gate before continuing. Build a working MVP before adding advanced memory, additional agents, dashboards or complex deployment.

Reference inspiration: Microsoft Customer Chatbot Solution Accelerator. This document describes a custom implementation using the selected stack; it is not a claim that the custom design is identical to Microsoft's original services or code.